WHEN FEATURES FAIL: WHY DEEP LEARNING SUCCEEDS WHERE TRADITIONAL ML CANNOT IN CHEST X-RAY CLASSIFICATION

GitHub Link: https://github.com/AkankshCaimi/AIDL-Pneumonia-Detection-from-Chest-XRay-Images-Using-Deep-Learning

1. INTRODUCTION

Pneumonia is one of the leading causes of death in children, especially under the age of 5. It was responsible for over 740,180 deaths of children in 2019 (WHO, 2022). This primarily happens due to limited number of trained radiologists in healthcare systems (Topol, 2019). This gap motivates the development of automated classification systems to support clinical decision making.

This project has been implemented using two deep learning models (Custom Convolutional Neural Network (CNN) and VGG16 Transfer Learning Model) and three traditional machine learning baseline models (Logistic regression, Support Vector Machines (SVM) and Random Forest). Kermany et al. (2018) dataset containing 5,856 labelled images was utilised for this project. This dataset had challenges such as a class imbalance of 2.88:1 and high variable image dimensions that have also been addressed. This dataset is publicly available at *https://data.mendeley.com/datasets/rscbjbr9sj/2*, licenced under CC by 4.0.

This report critically examines the design decisions, evaluates why certain approaches failed, and explores how emerging technologies could enhance the solution.

2. TRADITIONAL MACHINE LEARNING VERSUS DEEP LEARNING

The main difference between traditional ML and deep learning is in how discriminative representations are formed. Traditional methods follow a two-stage pipeline where domain experts design hand-crafted feature extractors to transform the raw input into fixed-dimensional vectors. These vectors are then fed to shallow classifiers (Goodfellow et al., 2016). Deep learning replaces this with end-to-end learning, jointly optimising feature extraction and classification through backpropagation to learn hierarchical representations directly from raw data (LeCun et al., 2015). CNNs further embed inductive biases such as local connectivity and translation equivariance that make them inherently suited to image data, as a pneumonia opacity is diagnostically relevant regardless of its position within lung fields. The following subsections empirically illustrate this paradigm difference.

2.1 The Feature Engineering Approach

For traditional ML models, a 1,812 dimensional feature vector was extracted per image. This dimensionality is a direct consequence of extraction parameters. It comprised Histogram of Oriented Gradients (HOG) descriptors for edge and shape patterns, Haralick texture features from Grey Level Co-occurrence Matrices (GLCM), Local Binary Pattern (LBP) histograms for micro texture and nine statistical intensity features. Each feature type targets specific radiographic characteristics as HOG captures structural edge patterns such as ribcage boundaries (Dalal and Triggs, 2005), Haralick features quantify textural changes from lung consolidation (Haralick et al., 2007), and LBP encodes micro-texture variations between healthy and pathological tissue (Ojala et al. 2002).

Three traditional ML models were selected deliberately to represent fundamentally different learning paradigms. Logistic Regression would test whether a linear decision boundary would suffice, SVM with RBF kernel would evaluate non-linear separation and Random Forest would provide ensemble-based piecewise partitioning along with importance ranking. The results revealed that all three traditional models had identical Pneumonia recall of 99.49% which indicates that the model predicted the majority class for every input. Whereas Normal recall was low (67%-74%), meaning healthy patients were wrongly diagnosed. These three models collapsed to identical majority class prediction which showed that limitation lay in hand-crafted features which even more sophisticated models like Gradient Boosting would face.

Although AUC-ROC scores showed a more nuanced story, Logistic regression, SVM and Random Forest achieved 0.9331, 0.9328 and 0.9262 respectively. This indicates that the features do encode meaningful discriminative information. AUC reveals that these models can rank pneumonia cases higher than normal cases with good accuracy. Hence the failure lies partially in features limited spatial ability and partially in the model's inability to establish an effective decision boundary under class imbalance. Even with **`class_weight='balanced'`**, the probability outputs remain skewed towards pneumonia because class weights adjust the loss function during training but do not shift the decision threshold. The model's probability output remains calibrated against the training distribution where the prior probability of Pneumonia is 74%. For ambiguous cases, outputs cluster above 0.5 rather than being centred on it, causing predictions to default to the majority class despite the reweighted loss.

The exploratory data analysis (EDA) showed that pixel intensity distribution between both classes overlapped (see Figure 1), and the average image difference is also subtle. Fixed descriptors operating at predetermined scales cannot adaptively attend to these region-specific patterns. Hence even the most important engineered features lacked spatial specificity needed for reliable classification.

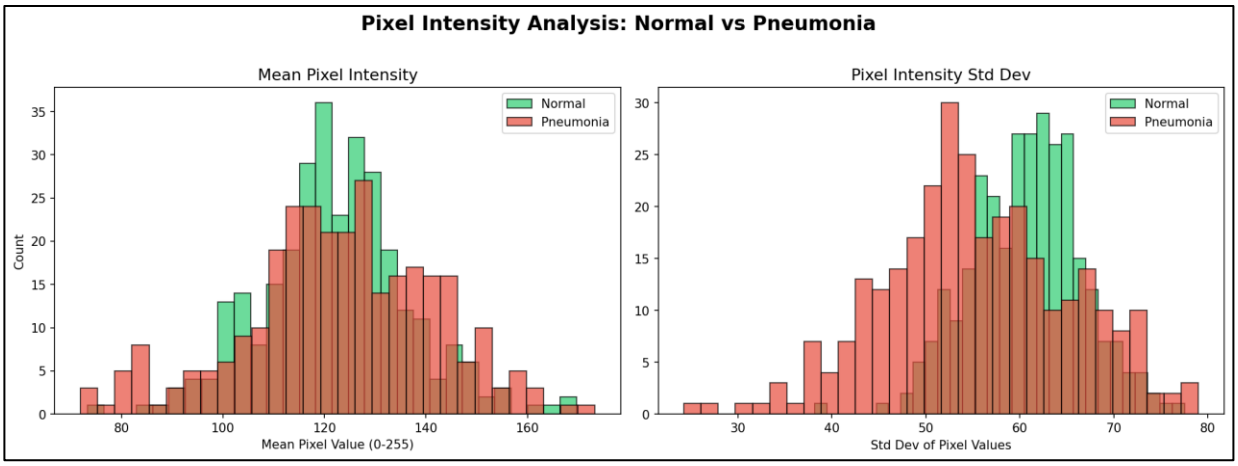


Figure 1

2.2 Why Deep Learning Succeeds

The results from training Custom CNN model revealed surprising results. The model had AUC score of 0.8949, underperforming all three traditional ML models. With only 4,446 training images the network did not have sufficient data to learn discriminative hierarchical representations from random initialisation. This aligns well with the findings of LeCun et al. (2015) that CNN models require large volumes of labelled data to learn effective representations.

The VGG16 transfer learning model resolved this problem by achieving highest AUC score of 0.9725, along with balanced precision (93.53%) and recall (96.41%). This model succeeded because pretrained ImageNet features encode universal low-level visual patterns such as edges, textures and gradients that transfer across domains (Yosinski et al., 2014).

Tajbakhsh et al. (2016) systematically compared CNNs trained from scratch versus fine-tuned pre-trained models across various medical imaging tasks and found that transfer learning consistently outperformed training from scratch. Raghu et al. (2019) also demonstrated benefits of transfer learning when datasets are small. The progression from Custom CNN to Transfer Learning was a deliberate choice because Custom CNN showed that learned features are superior in principle, but they required more data, hence, transfer learning solved this issue by providing pretrained representations from 1.2 million ImageNet images. Alternative approaches such as ResNet50 would face same data scarcity bottleneck, while Generative Augmentation Methods (GANs) would add complexity without guaranteed quality.

This shows that deep learning has advantages, but it is conditional, hence transfer learning overcomes the data limitations that caused other approaches to fail.

3. CRITICAL EVALUATION OF DESIGN DECISIONS

3.1 Data Preparation and Augmentation Strategy

The original dataset had a validation split that contained only 16 images, which is statistically inadequate for early stopping or hyperparameter selection. So, it was merged with training dataset and then split into 85/15 using stratified sampling. This yielded 786 validation images that provided enough statistical power while also preserving class distribution.

Stratified sampling preserved the 2.88:1 class ratio across both splits, preventing misleading early stopping signals. K-fold cross validation would provide more robust estimation but was impractical due to 40-50 minutes of training times per fold.

Images were resized to 224x224 pixels for VGG16 (Simonyan and Zisserman, 2014) and applied to all deep learning models for fair comparison. Though higher resolutions would preserve more diagnostic detail, the computational cost and overfitting risk would increase substantially.

Data Augmentation was applied to training data to reduce overfitting, and each augmentation was selected for clinical validity. Rotation (±15°) simulates patient positioning variation, width/height shifts (10%) account for centring differences, zoom (±10%) reflects varying patient-to-detector distances, horizontal flip exploits chest bilateral symmetry and brightness variation (±15%) addresses different machine calibrations.

3.2 Addressing Class Imbalance

The dataset had 2.88:1 Pneumonia-to-Normal ratio. To tackle this imbalance, class weight was applied (1.94x for Normal and 0.67x for Pneumonia) rather than oversampling because duplicating minority images increased the risk of overfitting. However, this proved insufficient for Custom CNN (Normal recall of 44%), suggesting that loss level correction alone cannot overcome strong distributional priors when the model has limited capacity.

3.3 Model Architecture Decisions

I. **Custom CNN:** The four block architecture (32→64→128→256 filters) was chosen because it was balanced between representational capacity and overfitting risk. Each block has dual convolutional layers with batch normalisation that stabilises training and reduces internal covariate shift (Ioffe and Szegedy, 2015). This is followed by max pooling and dropout (0.25) for regularisation (Srivastava et al., 2014). In total, this architecture contains 8 convolutional layers and 3 dense layers (see Appendix D for architecture summary).

II. **VGG16 Transfer Learning:** Fine-tuning transfer learning was specifically chosen over feature extraction. The domain gap between ImageNet's natural images and medical X-rays means that later VGG16 layers encode ImageNet specific features irrelevant to

pneumonia detection. Fine tuning adapts these layers to learn medical domain specific patterns while preserving universal early layer features. This was selected over ResNet50 and DenseNet121 because ResNet50 adds architectural complexity which was unnecessary for a binary task with limited fine-tuning data. And DenseNet121 has higher parameter interdependence through dense connectivity that makes fine tuning less predictable with small dataset. However, DenseNet121 was used in CheXNet (Rajpurkar et al., 2017) that achieved radiologist level of pneumonia detection. VGG16's sequential architecture allows more controlled layer-by-layer unfreezing. Rajaraman et al. (2018) compared VGG16, ResNet50 and others for medical image classification and found that VGG16 performed competitively on smaller datasets with reduced overfitting during fine-tuning. Though VGG16 has 138 million parameters, the choice prioritised fine-tuning controllability and proven medical imaging performance over architectural efficiency. It was employed in 2 phases. Phase 1 froze all VGG16 layers and trained only classification head which allowed adaptation to pretrained features without disturbing them. Phase 2 unfroze the final convolutional block with a decreased learning rate (tenfold) to prevent catastrophic forgetting (see Figure 2) (Yosinski et al., 2014). Only the last convolutional block was unfrozen because early layers encode universal transferable features while later layers require domain-specific adaptation (see Appendix C for two-phase training details). The validation loss spike at the phase transition and its recovery are analysed in Appendix G.

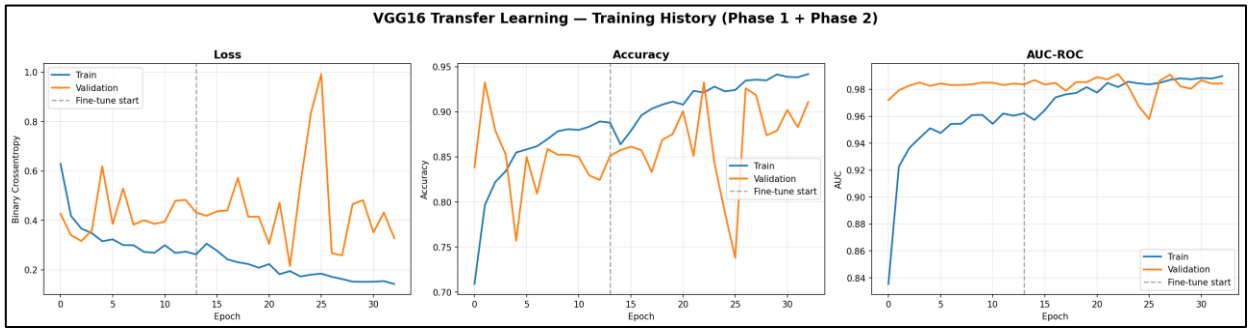


Figure 2

3.4 Evaluation Strategy

AUC-ROC was chosen because accuracy is misleading for imbalanced datasets, predicting all inputs as pneumonia achieves 74% accuracy while being clinically useless. AUC provides threshold-independent evaluation across all operating points (Litjens et al., 2017). Precision, recall and F1 were additionally tracked as false positive and false negative rates are critical in medical contexts.

3.5 Training Strategy

Early stopping monitored validation AUC (patience=10) rather than validation loss, as AUC is threshold-independent and more reliable for imbalanced data. **ReduceLROnPlateau(factor=0.5, patience=5)** halved the learning rate on AUC plateaus, allowing the optimiser to escape local minima before early stopping terminated training. Adam optimiser (Kingma and Ba, 2014) was chosen over SGD for its adaptive per-parameter learning rates and faster convergence on small datasets.

4. CRITICAL EVALUATION OF RESULTS

4.1 Performance Pattern Across Models

The VGG16 transfer learning model achieved the strongest overall performance with an accuracy of 93.59%, F1-score of 94.95% and AUC-score of 0.9725. But comparative analysis across all five models reveals insights beyond raw metrics (see Appendix A for full comparison table).

The Custom CNN had a peculiar failure pattern, it had only 6 false negatives (FN) but 132 false positives (FP) (see Figure 3). This means that the model misclassified over 56% of Normal cases as Pneumonia. This was due to over sensitive interactions between class imbalance and insufficient representational capacity. Despite class weighting, the network lacked the power to confidently identify Normal cases and predicted the statistically safer pneumonia class.

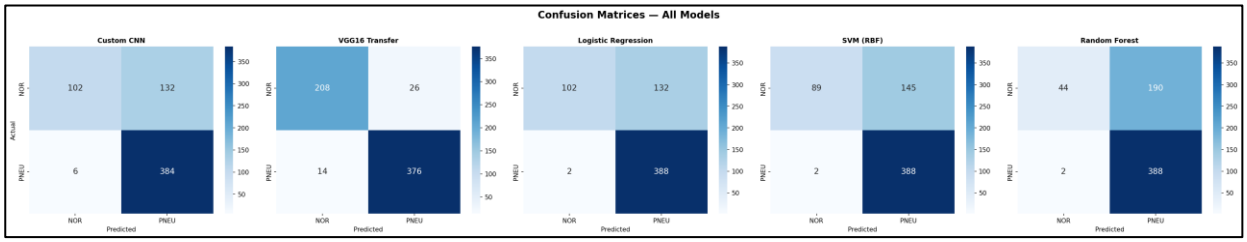


Figure 3

On the other hand, VGG16 had a Normal recall of 89% and Pneumonia recall of 96.41%. The pretrained features provided a richer representational foundation that enabled the classification head to learn more nuanced decision boundaries.

4.2 Clinical Interpretation of Errors

From clinical perspective, false negatives (missed pneumonia cases) have greater risk than false positives (wrongly diagnosed pneumonia case). VGG16's 14 false negatives out of 390 pneumonia cases shows a miss rate of 3.59%. While no automated system should operate without clinical oversight, this performance could support triage in high volume situations.

However, the 11.11% false positive rate would translate to significant unnecessary healthcare utilisation at population scale, making threshold optimisation essential for deployment.

4.3 Traditional ML Paradox

The AUC scores (0.926-0.933) of traditional ML models exceed the Custom CNN (0.895), but they are clinically unusable (see Figure 4). This is because clinical utility depends on performance at a specific operating point. AUC measures a model's ability to rank cases correctly across all thresholds, but clinical deployment requires performance at a single threshold. The traditional models rank well because the features encode meaningful discriminative information, but the feature space lacks sufficient separability to support a clean decision boundary at any individual threshold. This results in majority-class collapse regardless of threshold selection. But VGG16 achieves both strong ranking (high AUC) and effective thresholding (balanced precision and recall).

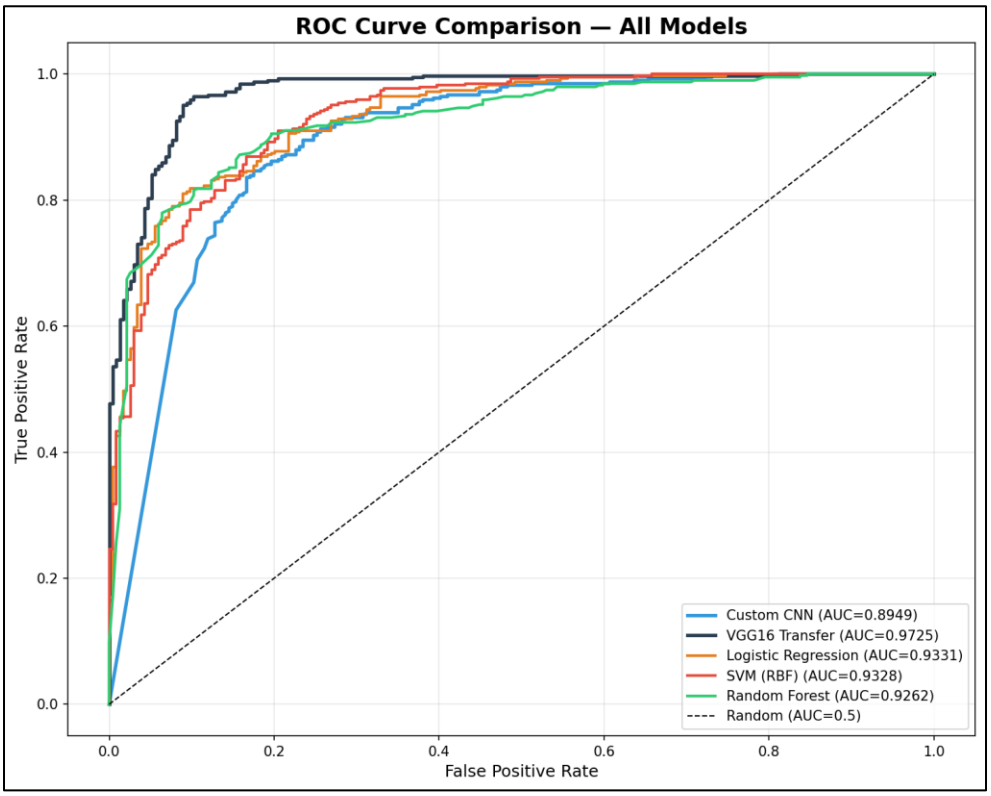


Figure 4

5. EXPLAINABILITY ANALYSIS

In medical AI, the explainability of models is essential where clinicians must understand why a model reaches a certain prediction (Topol, 2019).

I. **Grad-CAM** (Selvaraju et al., 2017) was applied to VGG16's block5_conv3 layer, the visualisations revealed clinically meaningful attention patterns. For true positive pneumonia cases the activation concentrated on lung fields, specifically in lower regions consistent with common pneumonia presentations. On the other hand, Custom CNN produced more scattered and less anatomically coherent maps showing weak learned

representations (see Figure 5). However, no radiologist validation was performed on these attention maps and without expert review, claims of clinical relevance remain unverified.

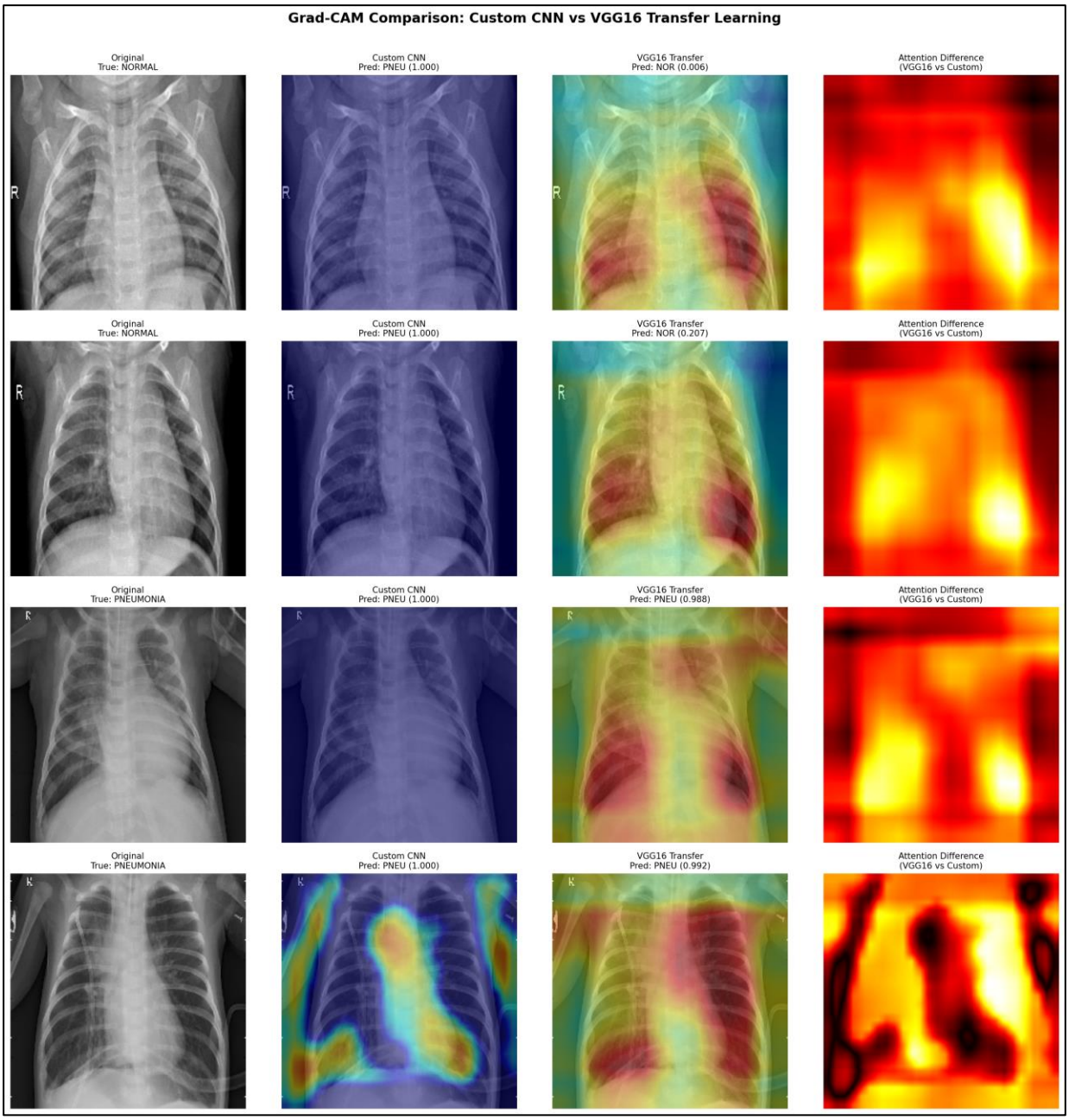


Figure 5

II. **SHAP's** (Lundberg and Lee, 2017) global summary plot identified HOG features as most influential while individual explanations showed which specific features drove particular prediction (see Figure 6). This offers a different explanatory modality compared to Grad-CAM. SHAP provides feature-level explanations but requires technical knowledge to interpret clinically.

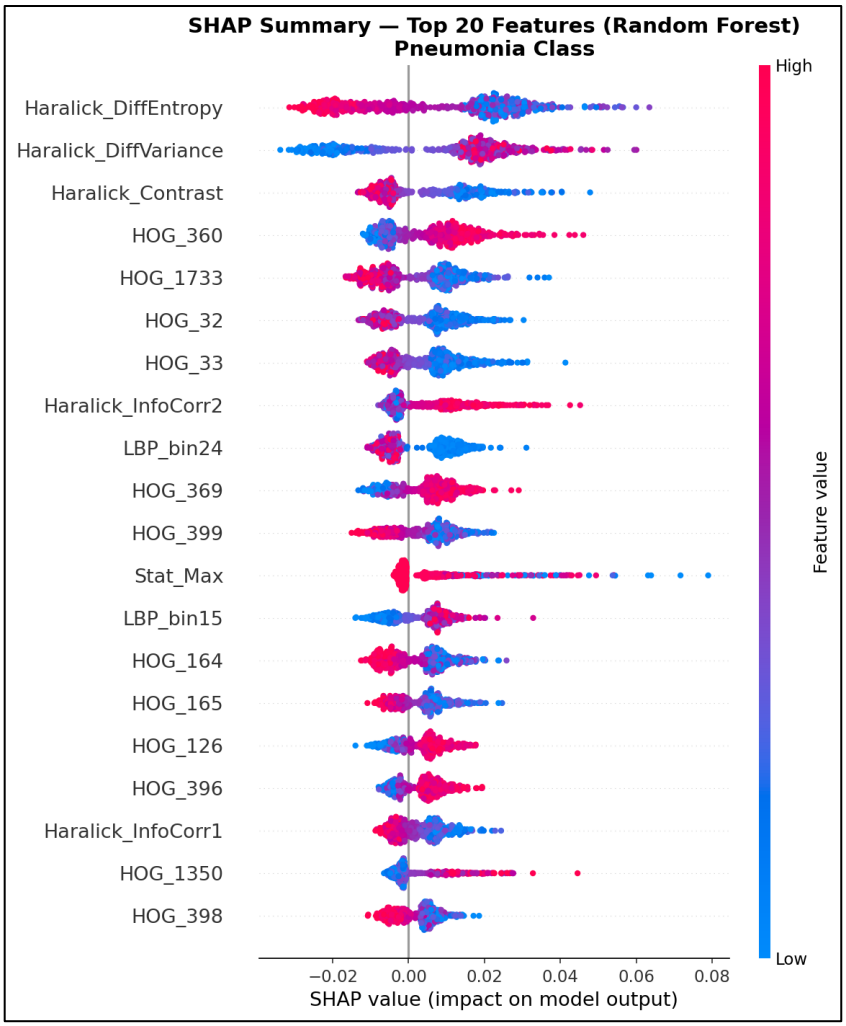


Figure 6

However, neither approach is without limitations. Grad-CAM provides post-hoc approximate explanations that are constrained by the final convolutional layer's spatial resolution, it cannot identify specific lesion boundaries. SHAP's exactness for tree models does not extend to neural networks where only approximate methods such as KernelSHAP are available which increase computational cost. This tension between spatial and feature based interpretability reflects a broader challenge in medical AI that remains as an active research area.

6. LIMITATIONS AND JUSTIFIED IMPROVEMENTS

6.1 Dataset Scale and Diversity

The dataset is small by deep learning standards (evident from Custom CNN's poor performance) and originates from a single institution (Guangzhou Women and Children's Medical Centre), encoding institution-specific protocols, demographics and scanner characteristics that limit generalisability.

6.2 Diagnostic Granularity

Binary classification oversimplifies clinical reality. Extending to multi-class classification distinguishing bacterial, viral and fungal aetiologies would directly influence treatment decisions.

6.3 Architectural Efficiency

EfficientNet-B0 (Tan and Le, 2019) achieves ImageNet accuracy with only 5.3 million parameters compared to VGG16's 138 million parameters. For binary classification task on small scale data, this architectural efficiency would drastically reduce computational cost, training time and overfitting risk.

6.4 Threshold Optimisation and Calibration

All models used the default 0.5 threshold, assuming equal misclassification costs. Clinical deployment requires task-specific threshold selection as different contexts (screening versus confirmatory diagnosis) demand different trade-offs. Additionally, probability calibration was not assessed to ensure confidence scores correspond to true likelihoods.

7. SOCIETAL IMPACT AND ETHICAL CONSIDERATIONS

This system could accelerate diagnosis in emergency departments where radiologist availability is limited. The VGG16 model could meaningfully support triage by flagging high probability cases for priority radiologist review. This ensures that the system accelerates and supports clinical decisions rather than replacing human expertise altogether.

The EU AI Act (2024) classifies medical diagnostic AI as high risk, mandating rigorous testing, transparency and human oversight. Similarly, FDA's framework for AI based Software as a Medical Device (SaMD) requires evidence of safety and effectiveness across intended patient populations.

Training VGG16 for this project represents modest energy consumption but production scale medical AI involves repeated retraining on datasets and multi modal ensembles which carry carbon cost and have impact on environment. Schwartz et al. (2020) advocates for "Green AI" practices which include reporting computational costs alongside the accuracy metrics.

Seyyed-Kalantari et al. (2021) demonstrated that chest X-ray models trained on demographically homogeneous datasets systematically underdiagnose underserved populations, with performance disparities correlating with race, sex and socioeconomic status. Deploying this model on adult patients, different ethnic groups or alternative imaging equipment would carry significant risk of biased outcomes.

Also, automation bias presents a risk where clinicians may over-rely on AI predictions, specifically for ambiguous cases. This risk is especially acute for edge cases where model confidence is

moderate. Displaying calibrated confidence intervals rather than binary predictions and requiring independent clinician assessment before viewing AI output would mitigate this risk (Topol, 2019).

8. FUTURE TRENDS AND OPPORTUNITIES

8.1 Vision Transformers

Vision Transformers (ViTs) (Dosovitskiy et al., 2020) process images as sequence of patches using self-attention mechanisms that can capture long range spatial dependencies that CNN may miss. However, ViTs typically require substantially more training data than CNNs due to the absence of inductive biases such as translation equivariance (Dosovitskiy et al., 2020). They also require more computational power.

Shamshad et al. (2023) found ViTs achieve state-of-the-art performance across medical imaging tasks when combined with appropriate pretraining. A further advantage is inherent explainability: self-attention weights show which image patches the model considers diagnostically relevant, potentially providing more faithful explanations than post-hoc Grad-CAM. A combined hybrid architecture that uses CNN layers for initial feature extraction and transformer layers for global reasoning will offer a practical middle ground applicable to this project's small dataset.

8.2 Self-Supervised Pretraining

Self-supervised methods such as Masked Autoencoders (He et al., 2022) pretrain on large volumes of unlabelled data by learning to reconstruct masked image regions. Fine-tuning this model on this project's labelled data could combine the advantages of domain specific pretraining (unlike ImageNet's VGG16) with the label efficiency of transfer learning. This combination may yield stronger performance than either approach alone.

Specifically, pretraining a MAE on large unlabelled chest X-ray repositories such as MIMIC-CXR (377,000 images) or CheXpert (224,000 images) would enable learning domain-specific representations of lung anatomy and pathological patterns without requiring any labels. This directly addresses both limitations of data scarcity and domain gap.

8.3 Federated Learning

Federated learning (McMahan et al., 2017) enables multiple institutions to collaboratively train a shared model without transferring patient data. Each hospital trains locally and shares only model weight updates with a central server. This would address the single-institution bias by incorporating training signals from hospitals across different demographics and imaging equipment, while complying with data protection regulations such as GDPR and HIPAA.

9. CONCLUSION

This project demonstrated that transfer learning using VGG16 outperformed both Custom CNN and traditional ML baselines for pneumonia detection. This insight does not mean that deep learning is inherently superior but that it has conditional advantages. The fundamental limitation of fixed feature representations for spatially nuanced medical images leads to systematic collapse of traditional ML models. This project's limitations define clear avenues for further improvement. Looking forward, the convergence of Vision transformers, Self-Supervised and Federated learning methods offer a compelling trajectory towards clinically deployable pneumonia detection systems that are more accurate, generalisable and equitable than the current approach.

REFERENCES

1. Dalal, N. and Triggs, B., 2005, June. Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)* (Vol. 1, pp. 886-893). *Ieee*.
2. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S. and Uszkoreit, J., 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
3. EU AI Act, 2024. Regulation (EU) 2024/1689 of the European Parliament and of the Council. *Official Journal of the European Union*.
4. Goodfellow, I., Bengio, Y. and Courville, A. 2016. *Deep learning* (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press.
5. Haralick, R.M., Shanmugam, K. and Dinstein, I.H., 2007. Textural features for image classification. *IEEE Transactions on systems, man, and cybernetics, (6),* pp.610-621.
6. He, K., Chen, X., Xie, S., Li, Y., Dollár, P. and Girshick, R., 2022. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16000-16009).
7. Ioffe, S. and Szegedy, C., 2015, June. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448-456). pmlr.
8. Kermany, D., Zhang, K. and Goldbaum, M. (2018) Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification. *Mendeley Data, V2. doi: 10.17632/rscbjbr9sj.2*.
9. Kingma, D.P. and Ba, J., 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
10. LeCun, Y., Bengio, Y. and Hinton, G., 2015. Deep learning. *nature, 521(7553)*, pp.436-444.

11. Litjens, G., Kooi, T., Bejnordi, B.E., Setio, A.A.A., Ciompi, F., Ghafoorian, M., Van Der Laak, J.A., Van Ginneken, B. and Sánchez, C.I., 2017. A survey on deep learning in medical image analysis. *Medical image analysis, 42*, pp.60-88.
12. Lundberg, S.M. and Lee, S.I., 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems, 30*.
13. McMahan, B., Moore, E., Ramage, D., Hampson, S. and y Arcas, B.A., 2017, April. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics* (pp. 1273-1282). *Pmlr*.
14. Ojala, T., Pietikainen, M. and Maenpaa, T., 2002. Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE Transactions on pattern analysis and machine intelligence, 24(7),* pp.971-987.
15. Raghu, M., Zhang, C., Kleinberg, J. and Bengio, S., 2019. Transfusion: Understanding transfer learning for medical imaging. *Advances in neural information processing systems, 32*.
16. Rajaraman, S., Antani, S.K., Poostchi, M., Silamut, K., Hossain, M.A., Maude, R.J., Jaeger, S. and Thoma, G.R., 2018. Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images. *PeerJ, 6,* p.e4568.
17. Rajpurkar, P., Irvin, J., Zhu, K., Yang, B., Mehta, H., Duan, T., Ding, D., Bagul, A., Langlotz, C., Shpanskaya, K. and Lungren, M.P., 2017. Chexnet: Radiologist-level pneumonia detection on chest x-rays with deep learning. *arXiv preprint arXiv:1711.05225*.
18. Schwartz, Roy, Jesse Dodge, Noah A. Smith, and Oren Etzioni, 2020. "Green ai." *Communications of the ACM 63*, no. 12 (2020): 54-63.
19. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision* (pp. 618-626).
20. Seyyed-Kalantari, L., Zhang, H., McDermott, M.B., Chen, I.Y. and Ghassemi, M., 2021. Underdiagnosis bias of artificial intelligence algorithms applied to chest radiographs in under-served patient populations. *Nature medicine, 27(12),* pp.2176-2182.
21. Shamshad, F., Khan, S., Zamir, S.W., Khan, M.H., Hayat, M., Khan, F.S. and Fu, H., 2023. Transformers in medical imaging: A survey. *Medical image analysis, 88,* p.102802.
22. Simonyan, K. and Zisserman, A., 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
23. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R., 2014. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research, 15(1)*, pp.1929-1958.
24. Tajbakhsh, N., Shin, J.Y., Gurudu, S.R., Hurst, R.T., Kendall, C.B., Gotway, M.B. and Liang, J., 2016. Convolutional neural networks for medical image analysis: Full training or fine tuning?. *IEEE transactions on medical imaging, 35(5)*, pp.1299-1312.

25. Tan, M. and Le, Q., 2019, May. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105-6114). PMLR.
26. Topol, E.J., 2019. High-performance medicine: the convergence of human and artificial intelligence. *Nature medicine, 25(1)*, pp.44-56.
27. World Health Organization, 2022. *Pneumonia Fact Sheet*. Available at: https://www.who.int/news-room/fact-sheets/detail/pneumonia.
28. Yosinski, J., Clune, J., Bengio, Y. and Lipson, H., 2014. How transferable are features in deep neural networks?. *Advances in neural information processing systems, 27*.

APPENDIX

Appendix A: Complete Model Comparison Table

Model	Type	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Training Time*
Custom CNN	Deep Learning	77.88%	74.42%	98.46%	84.77%	0.8949	44.6 min
VGG16 Transfer	Deep Learning	93.59%	93.53%	96.41%	94.95%	0.9725	50.7 min
Logistic Regression	Traditional ML	78.53%	74.62%	99.49%	85.27%	0.9331	0.5 sec
SVM (RBF)	Traditional ML	76.44%	72.80%	99.49%	84.07%	0.9328	16.6 sec
Random Forest	Traditional ML	69.23%	67.13%	99.49%	80.17%	0.9262	4.6 sec

**All training times are recorded using Apple M3 (8GB RAM) utilising the Metal GPU backend*

Appendix B: Key Code Excerpts

```
# Training: WITH clinically appropriate augmentation
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    brightness_range=[0.85, 1.15],
    fill_mode="nearest",
)
```

B.1: Data Augmentation Configuration

```python
# — 4.1 Build Custom CNN Architecture —

def build_custom_cnn(input_shape=(224, 224, 3)):
    """
    Custom CNN with 4 convolutional blocks.
    Designed for binary classification of chest X-rays.
    """
    model = models.Sequential(name="Custom_CNN")

    # Block 1: Low-level features (edges, simple textures)
    model.add(layers.Conv2D(32, (3, 3), activation="relu",
                            padding="same", input_shape=input_shape))
    model.add(layers.BatchNormalization())
    model.add(layers.Conv2D(32, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(0.25))

    # Block 2: Mid-level features (corners, basic shapes)
    model.add(layers.Conv2D(64, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.Conv2D(64, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(0.25))

    # Block 3: Higher-level features (lung field patterns)
    model.add(layers.Conv2D(128, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.Conv2D(128, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(0.25))

    # Block 4: Abstract features (disease-level patterns)
    model.add(layers.Conv2D(256, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.Conv2D(256, (3, 3), activation="relu", padding="same"))
    model.add(layers.BatchNormalization())
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Dropout(0.25))

    # Classification Head
    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(512, activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(128, activation="relu"))
    model.add(layers.Dropout(0.3))
    model.add(layers.Dense(1, activation="sigmoid"))

    return model


model_custom = build_custom_cnn(IMG_SHAPE)
model_custom.summary()
```

B.2: Custom CNN Architecture

```python
# — 5.1 Build Transfer Learning Model —

def build_transfer_model(input_shape=(224, 224, 3)):
    """
    VGG16 with frozen base + custom classification head.
    ImageNet weights provide pre-learned feature extractors.
    """
    base_model = VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=input_shape,
    )

    # Freeze all base layers for Phase 1
    base_model.trainable = False

    model = models.Sequential(name="VGG16_Transfer")
    model.add(base_model)
    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(256, activation="relu"))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.5))
    model.add(layers.Dense(128, activation="relu"))
    model.add(layers.Dropout(0.3))
    model.add(layers.Dense(1, activation="sigmoid"))

    return model, base_model


model_transfer, base_model_vgg = build_transfer_model(IMG_SHAPE)
model_transfer.summary()
```

B.3: VGG16 Transfer Learning Model

```python
base_model_vgg.trainable = True

# Freeze everything BEFORE block5 (layers 0–14)
for layer in base_model_vgg.layers[:15]:
    layer.trainable = False
```

B.4: Phase 2 Layer Unfreezing

```python
# — 3.4 Compute Class Weights —

# Design Decision: Class weights over oversampling because:
#   - Oversampling duplicates minority images → overfitting risk
#   - Class weights adjust loss function → balanced learning signal
#   - Combined with augmentation → effective imbalance handling

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0, 1]),
    y=train_generator.classes,
)
class_weight_dict = {0: class_weights[0], 1: class_weights[1]}
```

B.5: Class Weight Computation

```python
# — 6.1 Feature Extraction Functions —

def extract_features(image_path, img_size=128):
    """
    Extract hand-crafted features from a single chest X-ray.

    Uses 128x128 (not 224x224) because:
    - HOG/LBP/Haralick don't benefit as much from high resolution
    - Reduces extraction time significantly
    - Standard practice in traditional CV pipelines
    """
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        return None
    img = cv2.resize(img, (img_size, img_size))

    features = []

    # 1. HOG — edge/shape information
    hog_feat = hog(
        img,
        orientations=9,
        pixels_per_cell=(16, 16),
        cells_per_block=(2, 2),
        feature_vector=True,
    )
    features.extend(hog_feat)

    # 2. Haralick — GLCM texture descriptors
    haralick_feat = mahotas.features.haralick(img).mean(axis=0)
    features.extend(haralick_feat)

    # 3. LBP histogram — local texture patterns
    lbp = local_binary_pattern(img, P=24, R=3, method="uniform")
    lbp_hist, _ = np.histogram(lbp.ravel(), bins=26, range=(0, 26), density=True)
    features.extend(lbp_hist)

    # 4. Statistical features — global intensity
    features.extend([
        img.mean(),
        img.std(),
        float(np.median(img)),
        float(np.percentile(img, 25)),
        float(np.percentile(img, 75)),
        float(img.min()),
        float(img.max()),
        float(pd.Series(img.ravel()).skew()),
        float(pd.Series(img.ravel()).kurtosis()),
    ])

    return np.array(features, dtype=np.float64)
```

B.6: Class Weight Computation

```python
callbacks_custom = [
    EarlyStopping(
        monitor="val_auc",
        patience=10,
        mode="max",
        restore_best_weights=True,
        verbose=1,
    ),
    ModelCheckpoint(
        os.path.join(MODELS_DIR, "custom_cnn_best.keras"),
        monitor="val_auc",
        mode="max",
        save_best_only=True,
        verbose=1,
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=5,
        min_lr=1e-7,
        verbose=1,
    ),
]
```

B.7: Early Stopping and Callbacks Configuration

```python
def gradcam_tape_only(model, img_array, target_layer_name):
    """
    Grad-CAM implementation using only GradientTape.
    Compatible with Keras 3 and Apple Metal GPU.
    """
    img_tensor = tf.cast(img_array, tf.float32)

    # Check if target layer is inside a nested model (e.g. VGG16 base)
    is_nested = False
    base_model = None
    for layer in model.layers:
        if hasattr(layer, 'layers'):
            for sub_layer in layer.layers:
                if sub_layer.name == target_layer_name:
                    is_nested = True
                    base_model = layer
                    break

    if is_nested:
        # VGG16 transfer model - target layer is inside base model
        extractor = Model(
            inputs=base_model.input,
            outputs=base_model.get_layer(target_layer_name).output,
        )

        with tf.GradientTape() as tape:
            conv_output = extractor(img_tensor)
            tape.watch(conv_output)

            x = base_model(img_tensor)
            for layer in model.layers[1:]:
                x = layer(x)
            prediction = x[:, 0]

        grads = tape.gradient(prediction, conv_output)

    else:
        # Flat Sequential model (Custom CNN)
        with tf.GradientTape() as tape:
            x = img_tensor
            conv_output = None

            for layer in model.layers:
                x = layer(x)
                if layer.name == target_layer_name:
                    conv_output = x
                    tape.watch(conv_output)

            prediction = x[:, 0]

        grads = tape.gradient(prediction, conv_output)

    # Process heatmap
    if grads is None or conv_output is None:
        print(f"  Warning: Using activation-only heatmap for {target_layer_name}")
        if conv_output is not None:
            heatmap = tf.reduce_mean(conv_output[0], axis=-1)
        else:
            return np.zeros((14, 14))
    else:
        pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
        conv_output_val = conv_output[0]
        heatmap = conv_output_val @ pooled_grads[..., tf.newaxis]
        heatmap = tf.squeeze(heatmap)

    heatmap = tf.maximum(heatmap, 0) / (tf.math.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy()
```

B.8: Grad-CAM Implementation

Appendix C: VGG16 Two-Phase training Details

Aspect	Phase 1 (Frozen Base)	Phase 2 (Fine-Tuning)
Trainable Layers	Classification head only	Block5 (layers 15-18) + head
Learning Rate	$1x10^{-4}$	$1x10^{-5}$ (10x reduction)
Epochs Run	14/30 (early stopped)	19/20
Training Time*	20.1 min	30.6 min
Purpose	Adapt head to pretrained features	Domain-specific fine-tuning

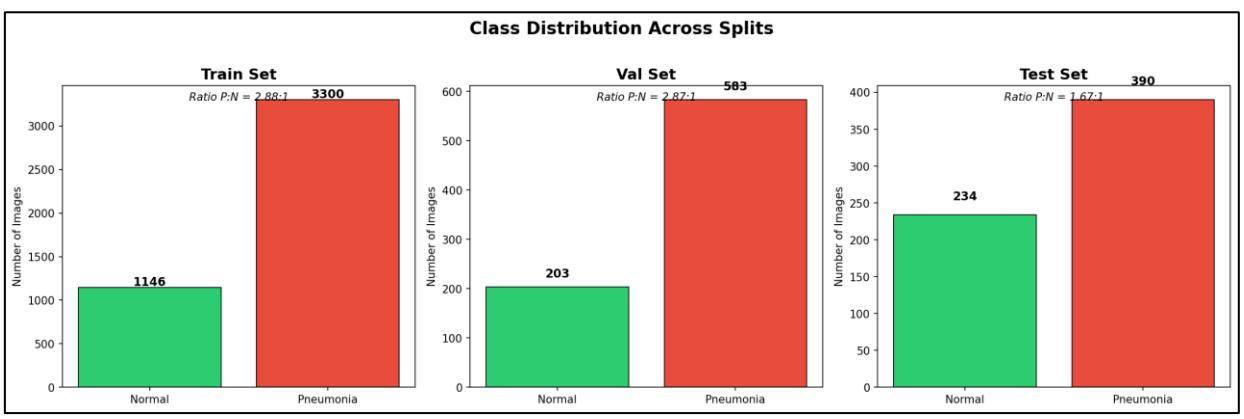
**All training times are recorded using Apple M3 (8GB RAM) utilising the Metal GPU backend*

Appendix D: Custom CNN Architecture Summary

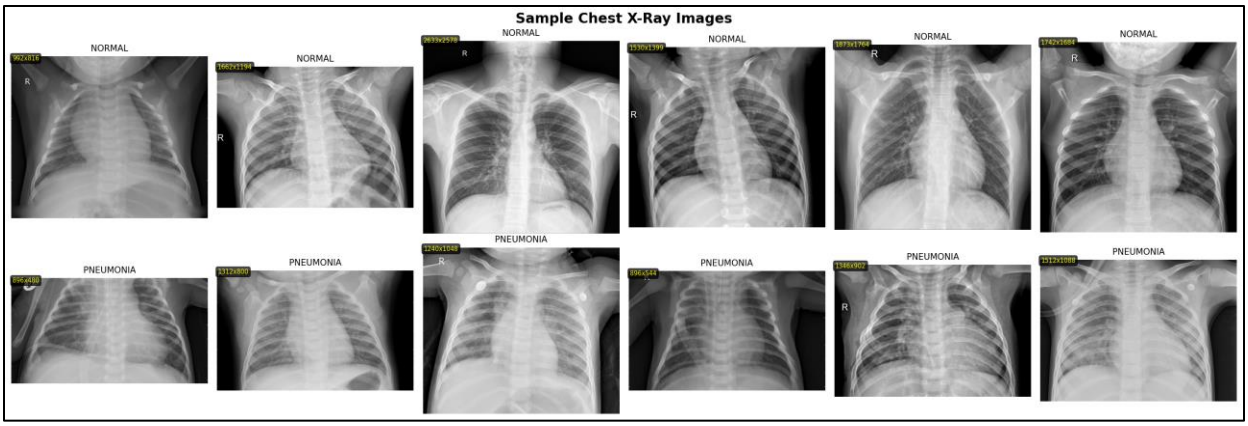
Block	Layers	Filters	Output Shape
Block 1	Conv2D→BN→Conv2D→BN→MaxPool→Dropout(0.25)	32	112x112x32
Block 2	Conv2D→BN→Conv2D→BN→MaxPool→Dropout(0.25)	64	56x56x64
Block 3	Conv2D→BN→Conv2D→BN→MaxPool→Dropout(0.25)	128	28x28x128
Block 4	Conv2D→BN→Conv2D→BN→MaxPool→Dropout(0.25)	256	14x14x256
Head	GAP→Dense(512)→BN→Dropout(0.5)→Dense(128)→Dropout(0.3)→Dense(1, sigmoid)	-	1

Total: 8 convolutional layers + 3 dense layers = 11 trainable layer groups

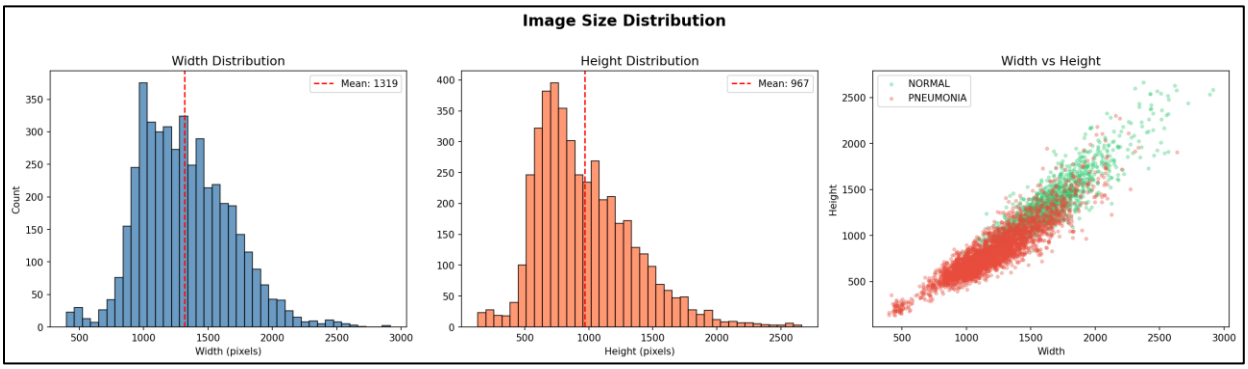
Appendix E: Code Output Images



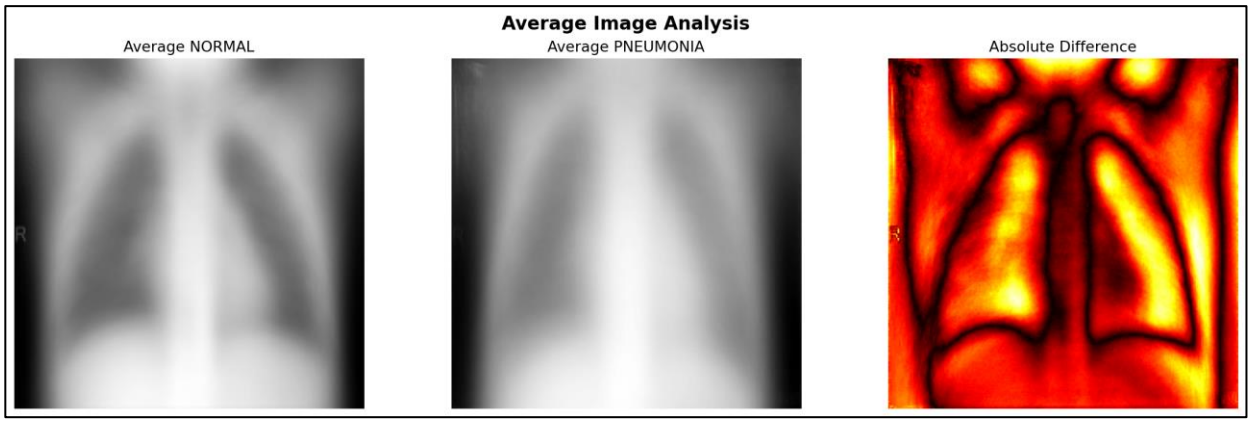
E.1: Class Distribution Across Splits



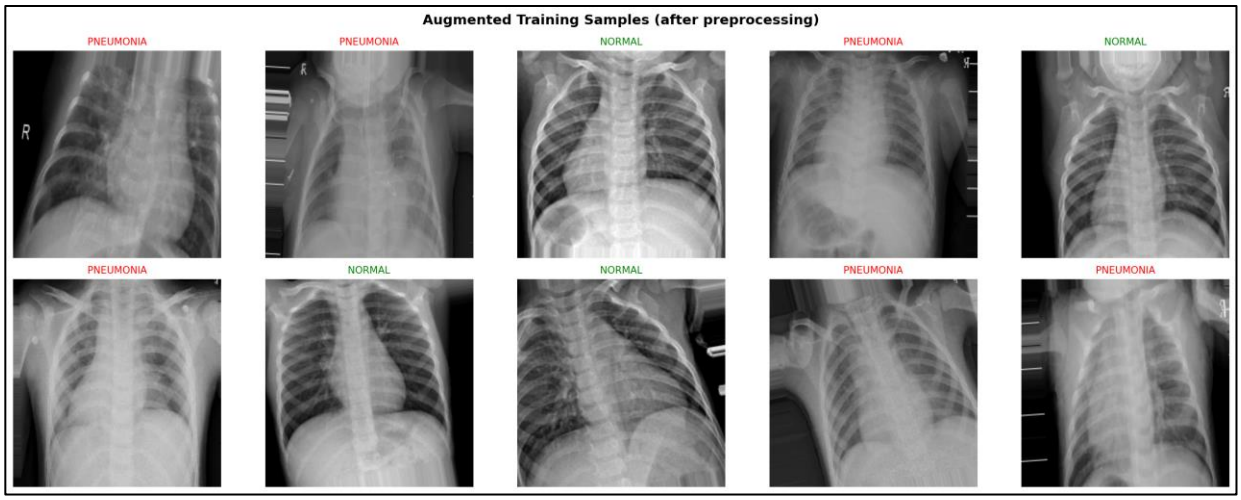
E.2: Sample Chest X-Ray Images



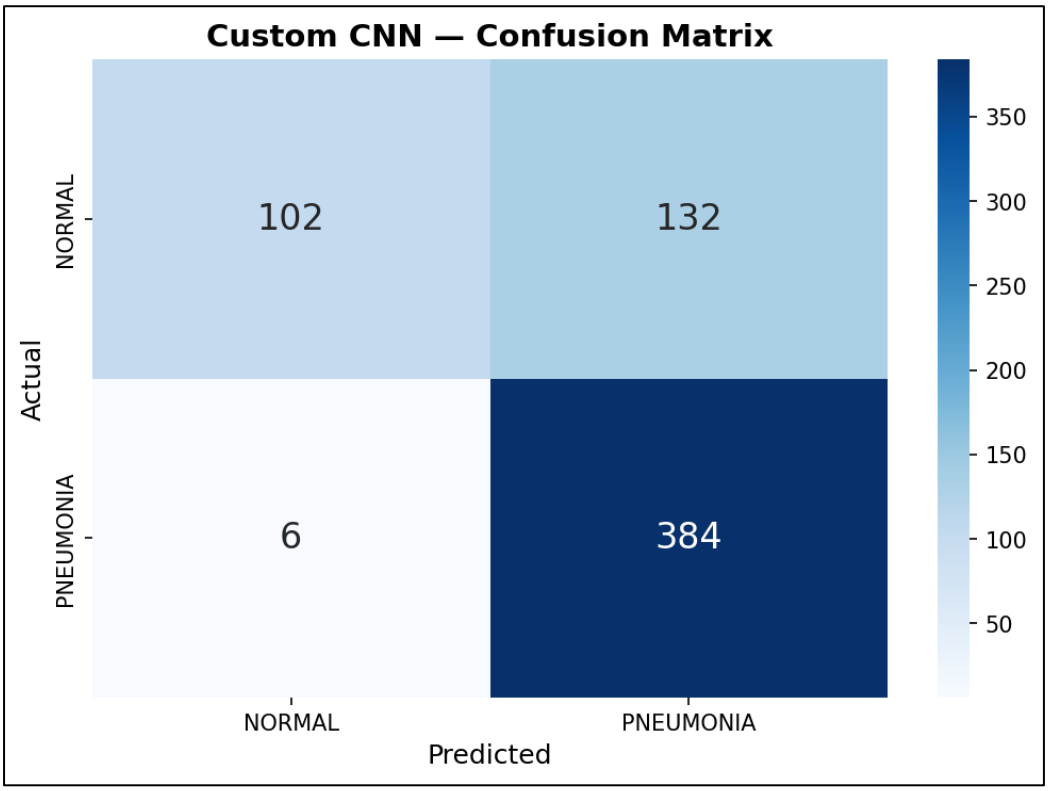
E.3: Image Size Distribution



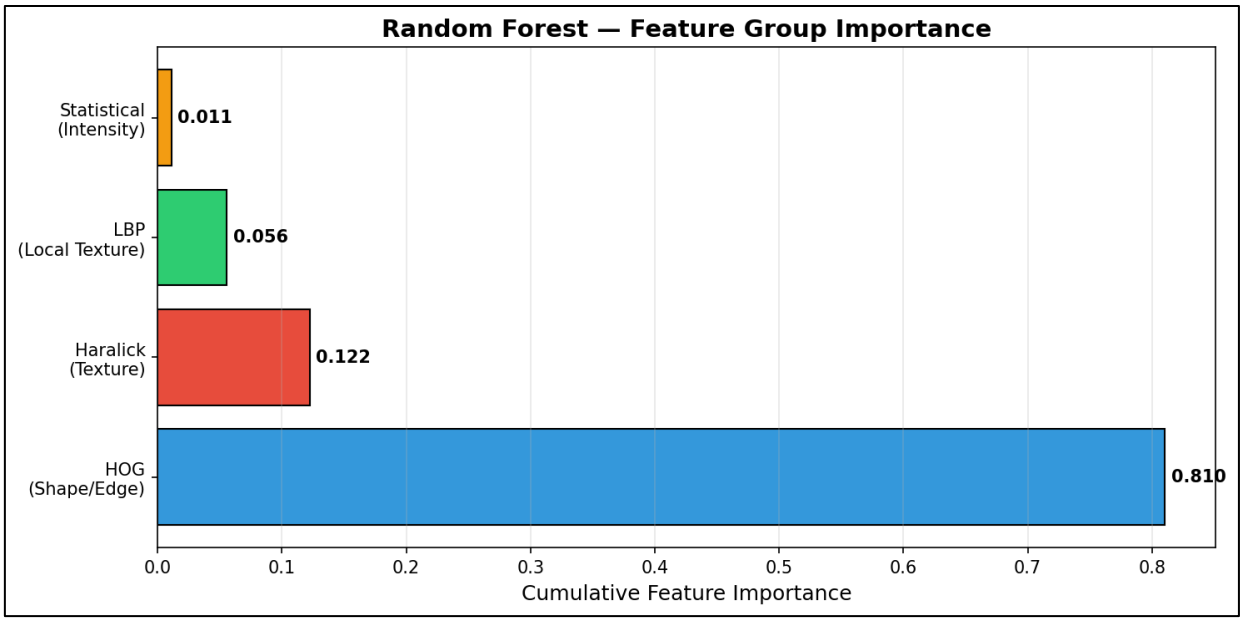
E.4: Average Images



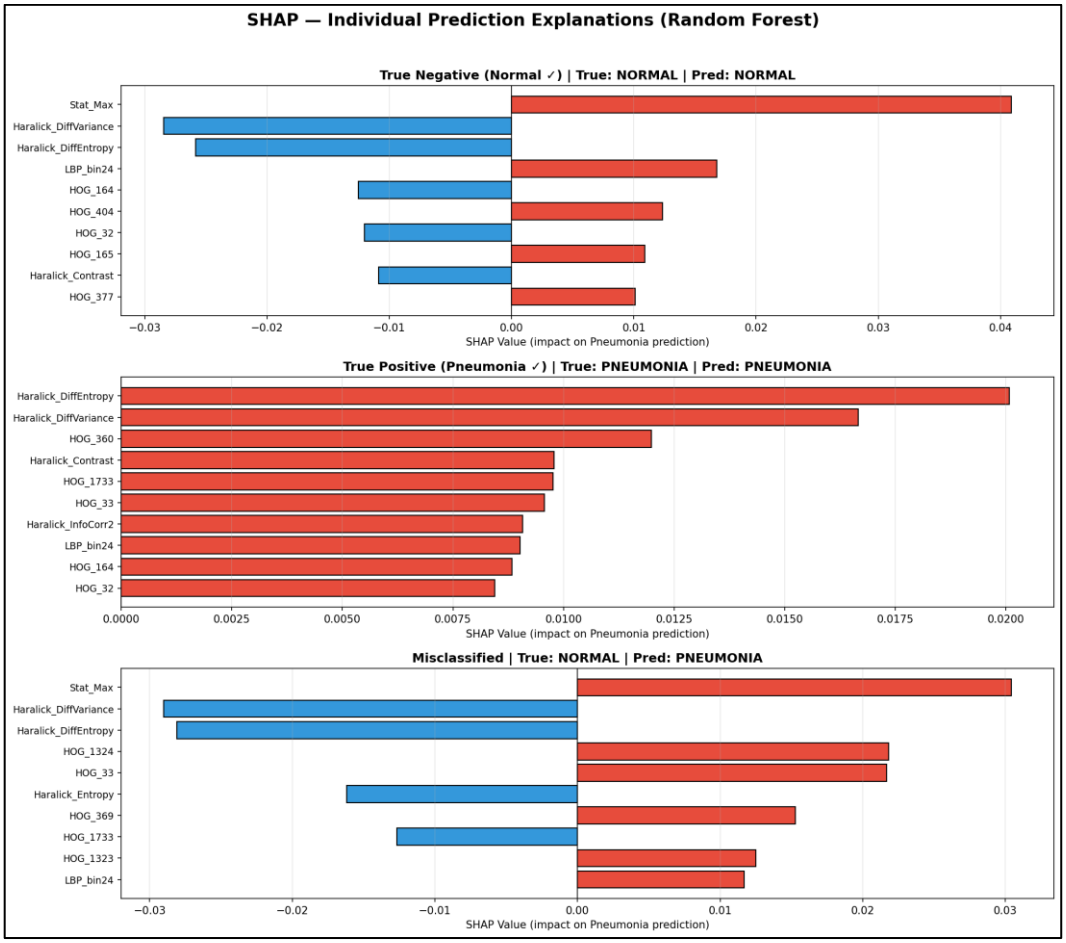
E.5: Augmented Training Samples



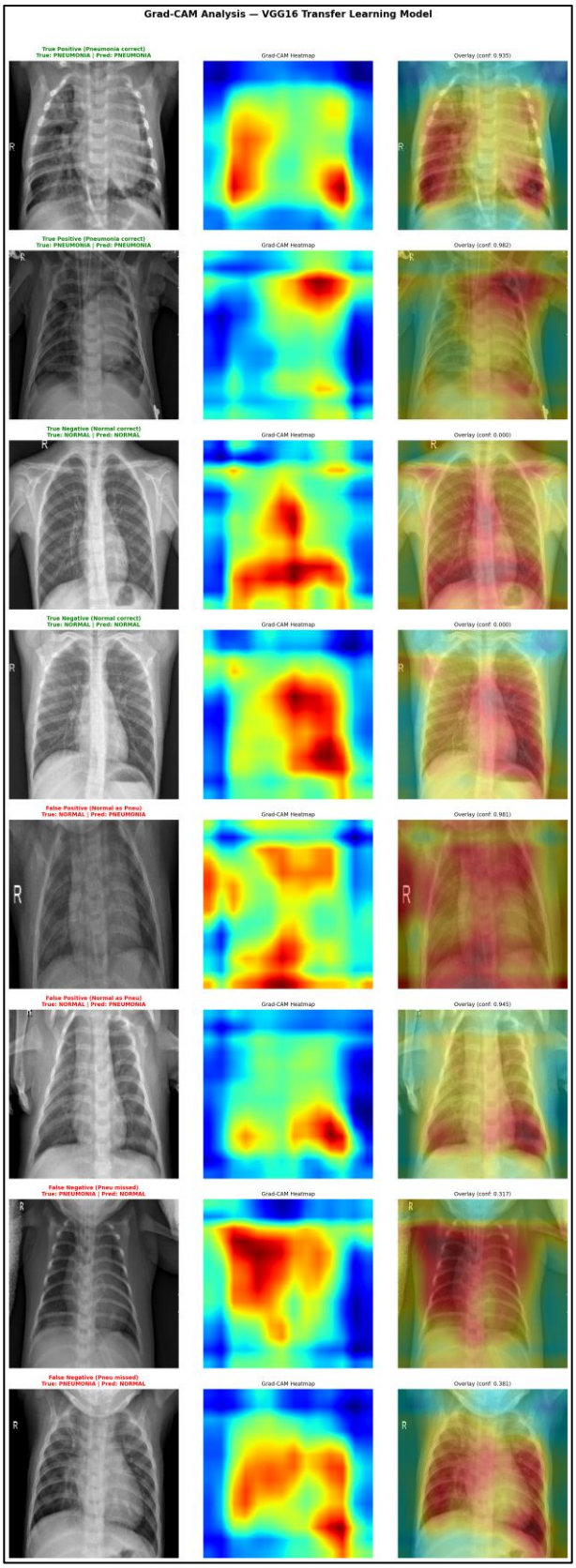
E.6: Custom CNN Confusion Matrix



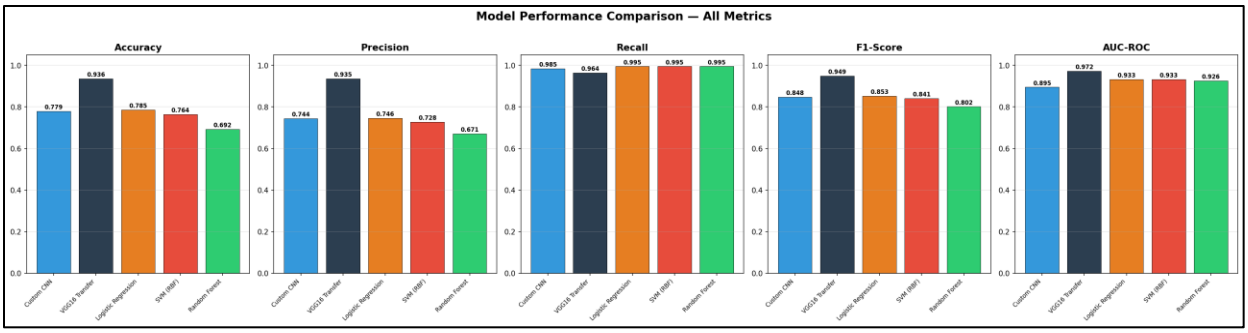
E.7: Feature Importance in Random Forest



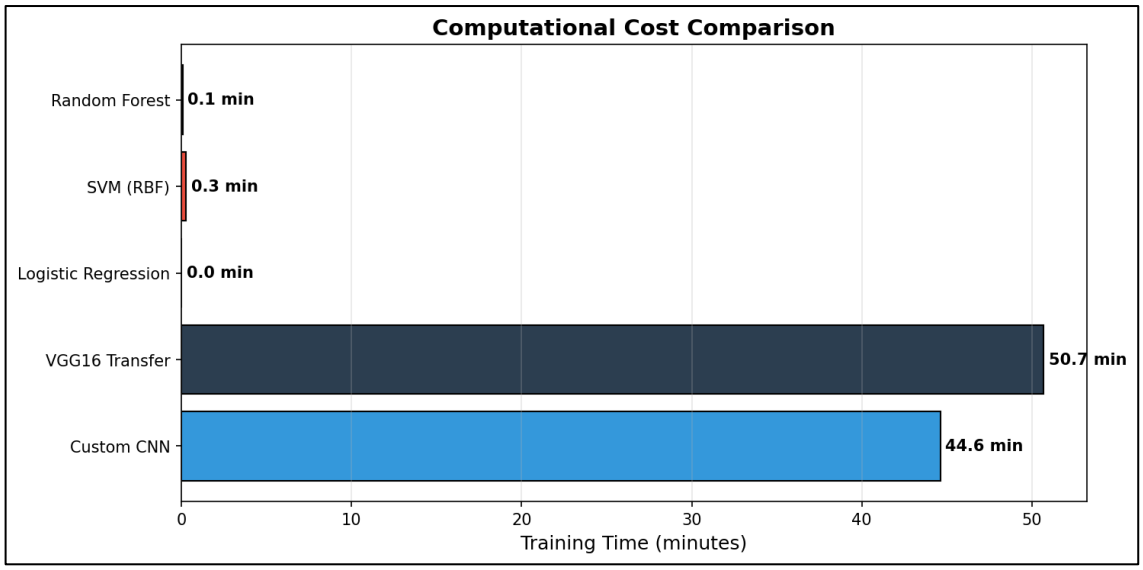
E.8: SHAP



E.9: Grad-CAM Analysis



E.10: Model performance Comparison



E.11: Training Times by using Apple M3 (8GB RAM) and utilising the Metal GPU backend

Appendix F: VGG16 Phase Transition Analysis

The visible spike in validation loss near the phase transition in Figure 2 is expected. When block5 is unfrozen, the newly trainable layers temporarily destabilise the learned feature representations, causing the classification head's decision boundaries to briefly misalign. The tenfold learning rate reduction in Phase 2 ensures recovery, with performance ultimately exceeding Phase 1 levels. A single-phase approach would fail because the randomly initialised classification head produces unstable gradients that would corrupt pretrained features through catastrophic forgetting.

Appendix G: Reflection on the Use of Generative AI

Throughout this project Claude and Gemini were used in supporting roles that include assisting with debugging compatibility issues between Keras 3 and Apple Metal GPU, clarifying API syntax for TensorFlow's GradientTape-based Grad-CAM implementation, providing guidance on structuring specific code sections and offering feedback on report clarity and organisation. The core arguments, design decisions and critical analysis presented in this report are my own. All AI-

generated outputs were cross-references against official documentation and validated for accuracy before incorporation.